SotilityTreasuryRouter.sol

SotilityTreasuryRouter.sol

Purpose:

A modular payout splitter that:

Accepts protocol revenue (ETH or SST)

Distributes to:

SOT holders (ETH dividends)

SST minting vault

SUG reward campaigns

Transparent, event-driven, and DAO-controlled. The Ecosystem financial autopilot — smart, composable, and DAO-tunable.

Integration Notes:

dividendPool → SOT dividend contract or distributor

stableVault → SST minting vault

campaignPool → treasury or SUG airdrop module

--

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/access/AccessControl.sol";
import "@openzeppelin/contracts/token/ERC20/IERC20.sol";

/**
 * @title SotilityTreasuryRouter
 * @dev Routes protocol revenue to SOT dividends, SST mint vaults, and SUG campaigns.
 */
contract SotilityTreasuryRouter is AccessControl {
    bytes32 public constant ROUTER_ROLE = keccak256("ROUTER_ROLE");

    address public dividendPool;
    address public stableVault;
    address public campaignPool;

    uint256 public splitToDividends = 4000; // 40%
    uint256 public splitToStable = 4000;    // 40%
    uint256 public splitToCampaigns = 2000; // 20%

    event RevenueReceived(address indexed sender, uint256 amount);
    event RevenueSplit(uint256 toDividends, uint256 toVault, uint256 toCampaigns);
    event DestinationsUpdated(address dividendPool, address stableVault, address campaignPool);
    event SplitsUpdated(uint256 toDividends, uint256 toVault, uint256 toCampaigns);

    constructor(address _dividendPool, address _stableVault, address _campaignPool) {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        dividendPool = _dividendPool;
        stableVault = _stableVault;
        campaignPool = _campaignPool;
    }

    receive() external payable {
        emit RevenueReceived(msg.sender, msg.value);
        _split(msg.value);
    }

    function _split(uint256 amount) internal {
        uint256 toDividends = (amount * splitToDividends) / 10000;
        uint256 toVault = (amount * splitToStable) / 10000;
        uint256 toCampaigns = amount - toDividends - toVault;

        payable(dividendPool).transfer(toDividends);
        payable(stableVault).transfer(toVault);
        payable(campaignPool).transfer(toCampaigns);

        emit RevenueSplit(toDividends, toVault, toCampaigns);
    }

    function updateDestinations(address _dividend, address _vault, address _campaign)
        external
        onlyRole(DEFAULT_ADMIN_ROLE)
    {
        dividendPool = _dividend;
        stableVault = _vault;
        campaignPool = _campaign;
        emit DestinationsUpdated(_dividend, _vault, _campaign);
    }

    function updateSplits(uint256 toDiv, uint256 toVault, uint256 toCamp)
        external
        onlyRole(DEFAULT_ADMIN_ROLE)
    {
        require(toDiv + toVault + toCamp == 10000, "Invalid split sum");
        splitToDividends = toDiv;
        splitToStable = toVault;
        splitToCampaigns = toCamp;
        emit SplitsUpdated(toDiv, toVault, toCamp);
    }
}
```

Notein
Scan the QR code to download Notein